OVERWATCH

LIVE COMPUTER-VISION OPERATOR CONSOLE · PROJECT REPORT

Sergio Arreola · Microsoft Software & Systems Academy · July 12, 2026

EXECUTIVE SUMMARY

Overwatch is a production-deployed computer-vision pipeline that monitors live public camera feeds — including two U.S.–Mexico border crossings in El Paso/Juárez — detects vehicles and people in each frame, and presents the results on a realtime operator console. It was designed and built as a working miniature of an intelligence, surveillance, and reconnaissance (ISR) workflow: unreliable external sensors in, cloud processing in the middle, and an operator-facing product out — with graceful degradation designed in at every layer.

Live demo: `https://overwatch-sarreola.azurewebsites.net` · **Source:** `https://github.com/sarreola07/overwatch` · **API docs:** `/swagger`

A frame captured by the pipeline from the Paso del Norte international bridge (El Paso/Juárez) HLS video stream — one of the live feeds the deployed system analyzes around the clock.

SYSTEM CAPABILITIES

Live multi-source ingestion	HLS video streams (border bridges, frames extracted via ffmpeg) and still-JPEG cameras (NYC DOT) polled concurrently; one dead feed never blocks the rest.
Pluggable inference	Three interchangeable detection backends behind one interface: local YOLOv8 on GPU (ONNX Runtime + DirectML, ~40 ms/frame), Azure AI Vision (cloud), and a mock for offline demos.
Realtime operator console	SignalR pushes every detection to connected dashboards the moment a frame is analyzed; clients fall back to HTTP polling automatically if the socket drops.
Live device cameras	Any visitor can stream their phone or laptop camera through the pipeline (GO LIVE); frames are analyzed and discarded, never stored.
Feed-health model	Every feed degrades Up → Stale → Down with data age always displayed; the health probe reports an outage only when all feeds are down.

ARCHITECTURE

1 · Ingest	Border-bridge HLS video (one keyframe per poll via ffmpeg) · NYC DOT still JPEG (HTTP) · visitor phone/laptop cameras (POST /api/analyze)
2 · Detect	IVisionClient — one interface, three backends: YoloVisionClient (local ONNX on GPU), AzureVisionClient (cloud API), MockVisionClient (offline)
3 · Store	DetectionStore — latest frame per feed, rolling detection history, feed-health state machine (Up / Stale / Down)
4 · Serve	REST (/api/*) for state · SignalR (/hubs/detections) pushes frame, feedFault, and liveDetections events to every connected operator console in realtime

TECHNOLOGY STACK

Layer	Technology	Notes
Backend	ASP.NET Core 8 — minimal APIs, BackgroundService, SignalR	Single deployable app; realtime push with polling fallback
Local inference	YOLOv8-small (COCO), ONNX Runtime, DirectML	~40 ms/frame on RTX 4080 SUPER; letterbox, tensor conversion, and NMS implemented in C#
Cloud inference	Azure AI Vision (Image Analysis 4.0, S1)	Used by the deployed site; global rate gate keeps all callers under quota
Video ingestion	ffmpeg (HLS keyframe extraction)	Static Linux build bundled in the cloud deployment
Frontend	Vanilla JS + CSS, SignalR client	Zero build step; custom instrument-console design system
Cloud	Azure App Service (Linux), Azure AI services	Resource group overwatch-rg; health-probe integrated
Delivery	GitHub + GitHub Actions CI, Swagger/OpenAPI, MIT license	Build verified on every push

PUBLIC API

Surface	Purpose
`GET /api/cameras`	Health and last-success time for every feed
`GET /api/frames/{id}`	Latest analyzed frame (base64 JPEG) with detections

Surface	Purpose
`GET /api/stats?minutes=10`	Rolling detection counts by label and feed
`POST /api/analyze`	Run any posted JPEG through the pipeline
`GET /healthz`	Liveness probe — 503 only if every feed is down
`WS /hubs/detections`	SignalR: frame, feedFault, liveDetections pushed realtime

ENGINEERING CHALLENGES SOLVED

Challenge	Resolution
API rate limiting (HTTP 429) the moment real Azure calls began	Global rate gate: all Vision callers share one queue with a configurable minimum gap (3.2 s on the free tier, 0.7 s on S1), so fixed cameras and live devices can never race the quota.
Cloud inference latency and per-call cost ceiling	Moved inference onto a local GPU behind the same interface — mock → Azure F0 → Azure S1 → local ONNX was a config change at each step, not a rewrite. 500–800 ms/frame became ~40 ms.
HLS video sources produce no still images	ffmpeg grabs one keyframe per poll (skip_frame nokey) piped to memory, capped at 25 s so a stalled stream degrades the feed instead of wedging the poll loop.
Bundled ffmpeg segfaulted on Azure App Service	Diagnosed via exit code 139 in the feed-health API; swapped static builds (johnvansickle → BtbN) and chmod the binary in the container startup command.
Windows-built zip failed Linux deployment	Deployment logs showed rsync failing on backslash entry paths from Compress-Archive; rebuilt archives with forward-slash paths.
Browsers served stale JavaScript after deploys	Versioned asset URLs busted the heuristic cache; caught by driving the UI after a deploy rather than assuming success.

RUN IT

```
git clone https://github.com/sarreola07/overwatch && cd overwatch && dotnet run
```

With no configuration it runs in mock mode (full pipeline, simulated detections). Add an Azure Vision key via user-secrets or drop a YOLOv8 ONNX model in models/ for real detection — steps in the README.